

ЛАБОРАТОРНА РОБОТА № 5

Docker: контейнеризація Express REST API та Docker Compose

Мета: набути практичних навичок контейнеризації Node.js-застосунку з Docker: multi-stage Dockerfile, .dockerignore, оркестрація Express + MongoDB через Docker Compose з volumes та healthcheck для моніторингу стану сервісів.

Хід роботи:

Завдання 1. Встановлення Docker

Встановлено Docker Desktop. Перевірено версії CLI та Compose, запущено тестовий контейнер hello-world для підтвердження роботи демона та доступу до Docker Hub.

```
PS C:\WINDOWS\system32> docker --version
Docker version 29.4.3, build 055a478
PS C:\WINDOWS\system32> docker compose version
Docker Compose version v5.1.3
```

Рис. 1. Вивід docker --version та docker compose version.

```
PS C:\WINDOWS\system32> docker run hello-world
Unable to find image 'hello-world:latest' locally
```

Рис. 2. Повідомлення Hello from Docker! після docker run hello-world.

Завдання 2. Dockerfile, .dockerignore та локальний запуск

Створено .dockerignore для виключення node_modules, dist, coverage, .env, тестів, скриптів звіту та файлів Docker з контексту збірки. Dockerfile реалізує multi-stage збірку: стадія builder (node:24) - npm ci та tsc; стадія production - лише production-залежності та dist з builder. Документовано порт EXPOSE 3000, процес запускається від USER node, CMD - node dist/src/server.js. Порядок інструкцій оптимізовано для кешування: спочатку package.json і package-lock.json, потім npm ci, далі вихідний код.

Лістинг .dockerignore:

```
node_modules
dist
coverage
.env
.git
.gitignore
*.md
```

					ДУ «Житомирська політехніка».26.121.20.000 – Лр5		
Змн.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Риженко Я.В.			Звіт з лабораторної роботи	Лім.	Арк.
Перевір.		Фуріхата Д.В.					1
Керівник						Аркушів	
Н. контр.						6	
Зав. каф.						ФІКТ Гр. ІПЗ-23-1	

```

*.docx
tests
scripts
jest.config.js
docker-compose*.yaml
compose.yaml
.dockerignore

```

Лістинг Dockerfile:

```

FROM node:24 AS builder

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci

COPY tsconfig.json ./
COPY src ./src
RUN npm run build

FROM node:24

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci --omit=dev

COPY --from=builder /app/dist ./dist

USER node

EXPOSE 3000

CMD ["node", "dist/src/server.js"]

```

Збірка та запуск з MongoDB Atlas (файл .env на хості):

`docker build -t myapp .`

`docker run -p 3000:3000 --env-file .env myapp`

```

PS D:\Work\Node.js\Lab05> docker build -t myapp .
[+] Building 1.2s (15/15) FINISHED
=> [internal] load build definition from Dockerfile                                docker:desktop-linux 0.0s
=> => transferring dockerfile: 397B                                             0.0s
=> [internal] load metadata for docker.io/library/node:24                       0.9s
=> [auth] library/node:pull token for registry-1.docker.io                     0.0s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 193B                                                0.0s
=> [builder 1/7] FROM docker.io/library/node:24@sha256:f22d6a1f082c02f292e86929b5b0442ac2e5eaf438a5dea9b1566601c3e05940 0.0s
=> => resolve docker.io/library/node:24@sha256:f22d6a1f082c02f292e86929b5b0442ac2e5eaf438a5dea9b1566601c3e05940 0.0s

```

Рис. 3. (Термінал - успішна збірка образу docker build -t myapp .)

		Риженко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – Лр5	Арк.
		Фуріхата Д.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		2

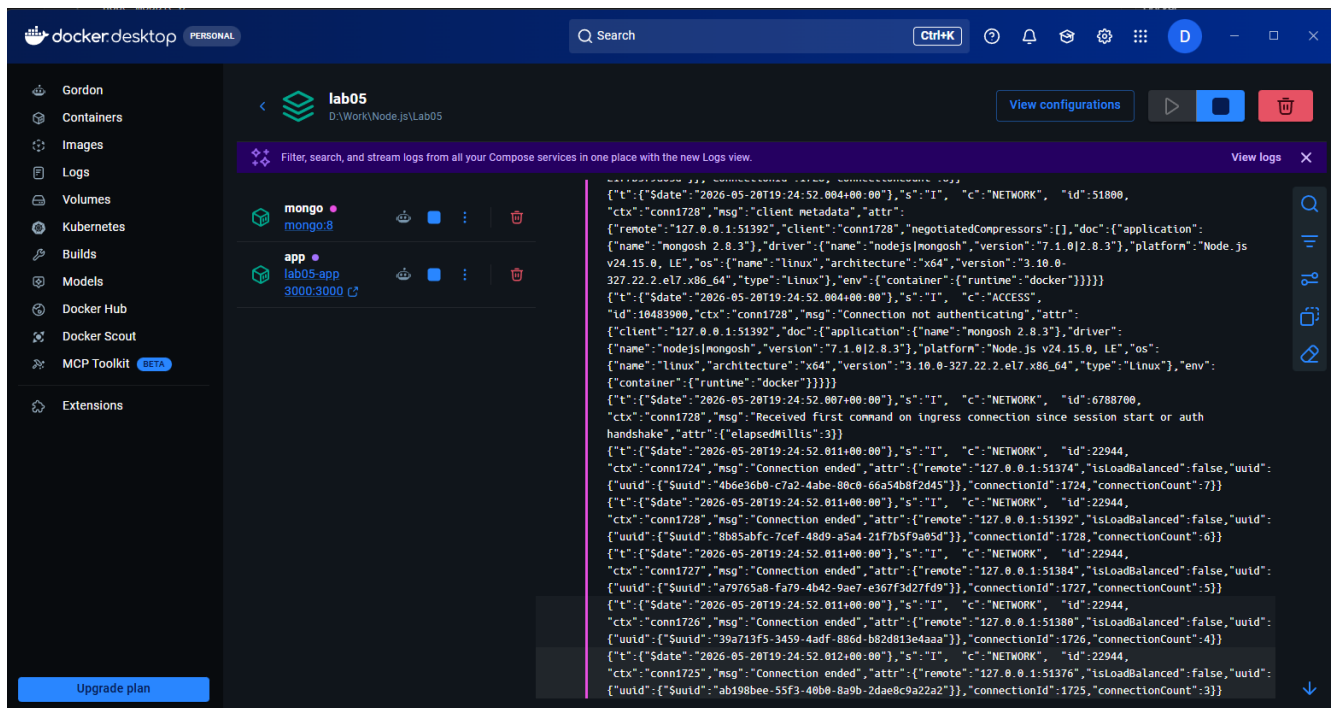


Рис. 4. Логи контейнера: підключення до MongoDB та <http://localhost:3000>

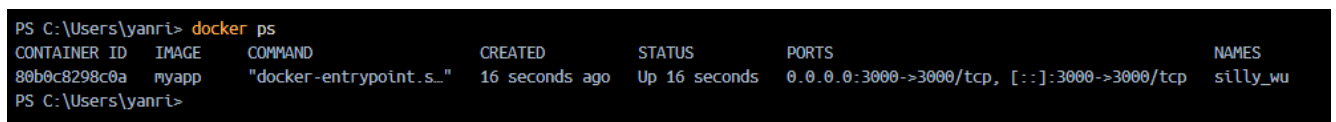


Рис. 5. docker ps (запущений контейнер myapp)

Завдання 3. Docker Compose

Створено compose.yaml з сервісами mongo (образ mongo:8, root-користувач, база mydb, volume mongo-data) та app (build: ., порт 3000:3000, MONGODB_URI на mongo, depends_on). Запуск стеку: `docker compose up --build`. Перевірено збереження даних після `docker compose down` та повторного `docker compose up`.

Лістинг compose.yaml:

```
services:
  mongo:
    image: mongo:8
    environment:
      MONGO_INITDB_ROOT_USERNAME: root
      MONGO_INITDB_ROOT_PASSWORD: example
      MONGO_INITDB_DATABASE: mydb
    volumes:
      - mongo-data:/data/db
    healthcheck:
      test: ['CMD', 'mongosh', '--eval', 'db.adminCommand("ping")']
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s
```

		Рижченко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – Лр5	Арк.
		Фуріхата Д.В.				3
Змн.	Арк.	№ докум.	Підпис	Дата		

```

app:
  build: .
  ports:
    - '3000:3000'
  depends_on:
    mongo:
      condition: service_healthy
  environment:
    NODE_ENV: production
    PORT: 3000
    MONGODB_URI: mongodb://root:example@mongo:27017/mydb?authSource=admin
  healthcheck:
    test:
      - CMD
      - node
      - -c
      - "fetch('http://127.0.0.1:3000/health').then((r)=>process.exit(r.ok?0:1)).catch(()=>process.exit(1))"
    interval: 15s
    timeout: 5s
    retries: 5
    start_period: 40s

volumes:
  mongo-data:

Команди:
docker compose up --build
docker compose ps
docker compose logs -f app
docker compose down
docker compose down -v

```

```

mongo-1 | {"t":{"$date":"2026-05-20T19:34:21.588+00:00"},"s":"I", "c":"NETWORK", "id":22943, "ctx":"listener","msg":"Connection accepted","attr":{"remote":
":127.0.0.1:60850","isLoadBalanced":false,"uuid":{"$uuid":{"$uuid":"dee01e1b-d455-4edf-a984-f2175eabf5e4"}}, "connectionId":73,"connectionCount":8}}
mongo-1 | {"t":{"$date":"2026-05-20T19:34:21.593+00:00"},"s":"I", "c":"NETWORK", "id":51800, "ctx":"conn73","msg":"client metadata","attr":{"remote":"127
.0.0.1:60850","client":"conn73","negotiatedCompressors":[],"doc":{"application":{"name":"mongosh 2.8.3"},"driver":{"name":"nodejs|mongosh","version":"7.1.0|2.
8.3"},"platform":"Node.js v24.15.0, LE","os":{"name":"linux","architecture":"x64","version":"3.10.0-327.22.2.el7.x86_64","type":"Linux"},"env":{"container":{"
runtime":"docker"}}}}}
mongo-1 | {"t":{"$date":"2026-05-20T19:34:21.593+00:00"},"s":"I", "c":"ACCESS", "id":10483900,"ctx":"conn73","msg":"Connection not authenticating","attr":
{"client":"127.0.0.1:60850","doc":{"application":{"name":"mongosh 2.8.3"},"driver":{"name":"nodejs|mongosh","version":"7.1.0|2.8.3"},"platform":"Node.js v24.1
5.0, LE","os":{"name":"linux","architecture":"x64","version":"3.10.0-327.22.2.el7.x86_64","type":"Linux"},"env":{"container":{"runtime":"docker"}}}}}
mongo-1 | {"t":{"$date":"2026-05-20T19:34:21.595+00:00"},"s":"I", "c":"NETWORK", "id":6788700,"ctx":"conn73","msg":"Received first command on ingress conn
ection since session start or auth handshake","attr":{"elapsedMillis":1}}
mongo-1 | {"t":{"$date":"2026-05-20T19:34:21.599+00:00"},"s":"I", "c":"NETWORK", "id":22944, "ctx":"conn69","msg":"Connection ended","attr":{"remote":"12
7.0.0.1:60826","isLoadBalanced":false,"uuid":{"$uuid":{"$uuid":"9d00f466-cf38-49be-9a09-7f730dc9aa7"}}, "connectionId":69,"connectionCount":7}}
mongo-1 | {"t":{"$date":"2026-05-20T19:34:21.600+00:00"},"s":"I", "c":"NETWORK", "id":22944, "ctx":"conn73","msg":"Connection ended","attr":{"remote":"12
7.0.0.1:60850","isLoadBalanced":false,"uuid":{"$uuid":{"$uuid":"dee01e1b-d455-4edf-a984-f2175eabf5e4"}}, "connectionId":73,"connectionCount":6}}
mongo-1 | {"t":{"$date":"2026-05-20T19:34:21.600+00:00"},"s":"I", "c":"NETWORK", "id":22944, "ctx":"conn72","msg":"Connection ended","attr":{"remote":"12
7.0.0.1:60842","isLoadBalanced":false,"uuid":{"$uuid":{"$uuid":"37185946-dede-4b53-8803-5e419583e304"}}, "connectionId":72,"connectionCount":5}}
mongo-1 | {"t":{"$date":"2026-05-20T19:34:21.600+00:00"},"s":"I", "c":"NETWORK", "id":22944, "ctx":"conn71","msg":"Connection ended","attr":{"remote":"12
7.0.0.1:60832","isLoadBalanced":false,"uuid":{"$uuid":{"$uuid":"37393376-28ae-470b-a308-e3b7ab813050"}}, "connectionId":71,"connectionCount":4}}
mongo-1 | {"t":{"$date":"2026-05-20T19:34:21.600+00:00"},"s":"I", "c":"NETWORK", "id":22944, "ctx":"conn70","msg":"Connection ended","attr":{"remote":"12
7.0.0.1:60828","isLoadBalanced":false,"uuid":{"$uuid":{"$uuid":"1f3311fd-949d-483b-8214-6e478393b0a0"}}, "connectionId":70,"connectionCount":3}}

```

Рис. 7. docker compose up --build, обидва сервіси Started

```

PS D:\Work\Node.js\Lab05> docker compose ps
NAME                IMAGE                COMMAND                  SERVICE    CREATED        STATUS               PORTS
lab05-app-1         lab05-app           "docker-entrypoint.s..." app        3 minutes ago   Up 2 minutes (healthy) 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
lab05-mongo-1       mongo:8             "docker-entrypoint.s..." mongo     3 minutes ago   Up 2 minutes (healthy) 27017/tcp

```

Рис. 8. (docker compose ps - сервіси app та mongo у стані running)

		Рижченко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Лр5	Арк.
		Фуріхата Д.В.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

```

>> curl -d '{"title":"Inception","releaseYear":2010,"genre":"sci-fi","director":"Christopher Nolan"}'
{"title":"Inception","releaseYear":2010,"genre":"sci-fi","director":"Christopher Nolan","createdAt":"2026-05-20T19:38:00.012Z","updatedAt":"2026-05-20T19:38:00.012Z","displayTitle":"Inception (2010)","id":"6a0e0d98053cbd6115f0ab23"}
PS D:\Work\Node.js\Lab05> curl.exe -s http://localhost:3000/api/movies
{"data":[{"title":"Inception","releaseYear":2010,"genre":"sci-fi","director":"Christopher Nolan","createdAt":"2026-05-20T19:38:00.012Z","updatedAt":"2026-05-20T19:38:00.012Z","displayTitle":"Inception (2010)","id":"6a0e0d98053cbd6115f0ab23"},{"title":"Inception","releaseYear":2010,"genre":"sci-fi","director":"Christopher Nolan","createdAt":"2026-05-20T19:37:48.646Z","updatedAt":"2026-05-20T19:37:48.646Z","displayTitle":"Inception (2010)","id":"6a0e0d8c053cbd6115f0ab22"}], "pagination":{"page":1,"limit":10,"total":2,"totalPages":1}}
PS D:\Work\Node.js\Lab05>

```

Рис. 9. POST /api/movies через curl - створення запису в локальній MongoDB

```

PS D:\Work\Node.js\Lab05> docker compose down
>>
[+] down 3/3
✓ Container lab05-app-1 Removed 0.4s
✓ Container lab05-mongo-1 Removed 0.3s
✓ Network lab05_default Removed 0.3s
PS D:\Work\Node.js\Lab05> docker compose up -d
[+] up 3/3
✓ Network lab05_default Created 0.0s
✓ Container lab05-mongo-1 Healthy 6.4s
✓ Container lab05-app-1 Started 6.6s

What's next:
  Filter, search, and stream logs from all your Compose services
  in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs
PS D:\Work\Node.js\Lab05>
>> curl.exe -s http://localhost:3000/api/movies
{"data":[{"title":"Inception","releaseYear":2010,"genre":"sci-fi","director":"Christopher Nolan","createdAt":"2026-05-20T19:38:00.012Z","updatedAt":"2026-05-20T19:38:00.012Z","displayTitle":"Inception (2010)","id":"6a0e0d98053cbd6115f0ab23"},{"title":"Inception","releaseYear":2010,"genre":"sci-fi","director":"Christopher Nolan","createdAt":"2026-05-20T19:37:48.646Z","updatedAt":"2026-05-20T19:37:48.646Z","displayTitle":"Inception (2010)","id":"6a0e0d8c053cbd6115f0ab22"}], "pagination":{"page":1,"limit":10,"total":2,"totalPages":1}}
PS D:\Work\Node.js\Lab05>

```

Рис. 10. Після docker compose down і повторного up - запис залишився (GET /api/movies)

Завдання 4. Healthcheck-ендпоінт

Додано GET /health у app.ts перед усіма middleware та маршрутами. Ендпоінт перевіряє mongoose.connection.readyState: 200 і status ok при підключенні, 503 - якщо БД недоступна. У compose.yaml налаштовано healthcheck для app (fetch /health) та mongo (mongosh ping), depends_on app очікує service_healthy для mongo.

Лістинг src/app.ts (фрагмент /health):

```

import cors from 'cors';
import express from 'express';
import mongoose from 'mongoose';
import { errorHandler } from './middleware/errorHandler';
import entityRouter from './routes/entity';

const app = express();

app.get('/health', (_req, res) => {
  const readyState = mongoose.connection.readyState;
  const connected = readyState === 1;
  res.status(connected ? 200 : 503).json({
    status: connected ? 'ok' : 'unavailable',
    database: {
      readyState,
      connected,
    },
  });
});

```

		Рижченко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – Пр5	Арк.
		Фуріхата Д.В.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

});
});

app.use(cors());
app.use(express.json());

PS D:\Work\Node.js\Lab05> curl.exe -s http://localhost:3000/health
{"status":"ok","database":{"readyState":1,"connected":true}}

```

Рис. 11. (curl http://localhost:3000/health - JSON зі status ok та database.connected)

```

#16 exporting config sha256:b58a4a11532dc076b30e19d984e5da3f535dc2c9f721c3df7fc1885f59670c9c done
#16 exporting attestation manifest sha256:e33c3e6de016235f5c6d9c8994ec3ea794d84edac62d5d6bb1953ed8dde4dc8a 0.0s done
#16 exporting manifest list sha256:97bfc278f7f37248b59484ffdbad456aade7dc13e4e5eb41dc82def16db8c399
#16 exporting manifest list sha256:97bfc278f7f37248b59484ffdbad456aade7dc13e4e5eb41dc82def16db8c399 0.0s done
#16 naming to docker.io/library/lab05-app:latest done
#16 unpacking to docker.io/library/lab05-app:latest 0.0s done
#16 DONE 0.1s

#17 resolving provenance for metadata file
#17 DONE 0.0s
[+] up 3/3
✓ Image lab05-app Built 1.9
✓ Container lab05-mongo-1 Healthy 1.0
✓ Container lab05-app-1 Started 1.2

What's next:
  Filter, search, and stream logs from all your Compose services
  in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs
PS D:\Work\Node.js\Lab05> Start-Sleep -Seconds 60
PS D:\Work\Node.js\Lab05> docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
lab05-app-1	lab05-app	"docker-entrypoint.s..."	app	2 minutes ago	Up 2 minutes (healthy)	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
lab05-mongo-1	mongo:8	"docker-entrypoint.s..."	mongo	3 minutes ago	Up 3 minutes (healthy)	27017/tcp

Рис. 12. (docker compose ps - обидва сервіси зі статусом (healthy))

Посилання на репозиторій GitLab - <https://github.com/DDXwild/node.js-practice>

Висновок: REST API з лабораторної роботи №4 розгорнуто в Docker-контейнері з multi-stage збіркою; Docker Compose піднімає Express і MongoDB однією командою, дані зберігаються у volume, стан сервісів контролюється через healthcheck. Проєкт готовий до здачі.

		Рижченко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – Лр5	Арк.
		Фуріхата Д.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		6